



西北工业大学

NORTHWESTERN POLYTECHNICAL UNIVERSITY

软件架构设计

主讲：李伟刚

liweigang@nwpu.edu.cn

西北工业大学软件学院



第二章 软件架构设计方法

主要内容

- 2.1 面向对象的概念
- 2.2 信息隐藏
- 2.3 继承和泛化/特化
- 2.4 并发处理
- 2.5 设计模式
- 2.6 软件架构和构件
- 2.7 软件质量属性
- 2.8 基于用例的软件生存周期
- 2.9 需求、分析和设计建模活动



2.1 面向对象的概念

● 历史

- **Simula 67: Dahl & Hoare(1967)**首次使用类和继承概念
- **信息隐藏: Parnas(1972)**引入的模块设计方法
- **Smalltalk:** 由美国施乐公司开发 (**smalltalk-72/-78/-80**) , 正式提出“面向对象”

● 核心

- 解决了软件适应与演化问题
- 使软件建模从面向对象的视角展开



2.1 面向对象的概念

● 对象

- 现实世界中 **物理的**或 **概念的**实体
- 面向对象的软件由对象构成
- 对象包含数据及作用于数据之上的过程
- 对象有唯一的标识

● 类

- 具有相同特征的对象集合

● 属性

- 对象所持有的一个数据值
- 类中的属性名是唯一的



2.1 面向对象的概念

● 操作

- 由一个对象所执行的一项功能的 **规约**
- 功能的实现称为 **方法**
- 操作和方法经常混用
- 一个操作的 **签名** 包括该操作的名字、参数以及返回值

● 接口

- 对象的接口是它提供的操作的集合，操作通过 **签名** 定义
- 对象的类型用接口定义
- 对象的实现用类来定义
- 类是一种 **抽象数据类型**



2.2 信息隐藏

● 思路

- 减少或完全消除全局数据
- 将信息隐藏作为软件系统分解成模块的一种标准
- 信息隐藏的模块实现了 **为变化而设计** 的目标

● 面向对象设计中的信息隐藏

- 对象中不需要被其他对象看到的那些部分被隐藏起来
- 可能发生变化的信息被 **封装**（隐藏）在对象中
- 通过操作来访问它们
- 操作的规约被称为对象的 **接口**
- 接口表示了对象的可见部分
- 其他对象通过接口调用操作



2.2 信息隐藏

● 益处

- 当改变那些被几个对象访问的重要的数据结构时

非信息隐藏

- 需要修改访问该数据结构的所有对象
- 会对所有与该数据结构相关的对象产生影响

信息隐藏

- 将与该数据结构相关的设计决策、其内部连接以及操纵该数据结构的操作细节隐藏起来
- 仅对包含该数据结构的对象产生影响
- 接口不变，所以其他对象不受影响



2.2 信息隐藏

● 例子

○ 几个模块访问一个 **栈**

非信息隐藏

- 模块是过程或函数
- 栈是全局数据结构
- 每个模块都直接访问栈，都知道栈的表示形式

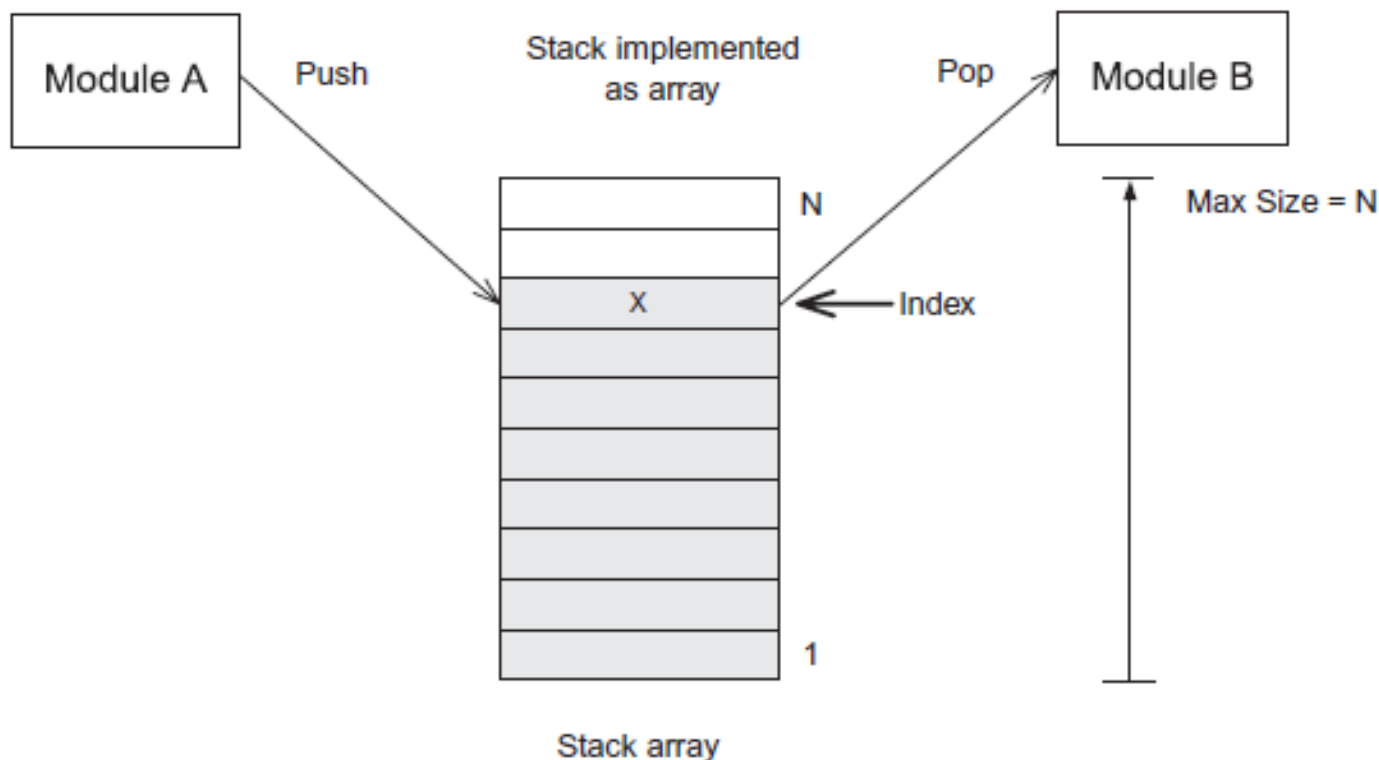
信息隐藏

- 模块是对象或构件
- 信息隐藏的栈对象
- 不允许其他对象直接访问栈对象内部的数据结构，通过调用栈对象的操作来访问



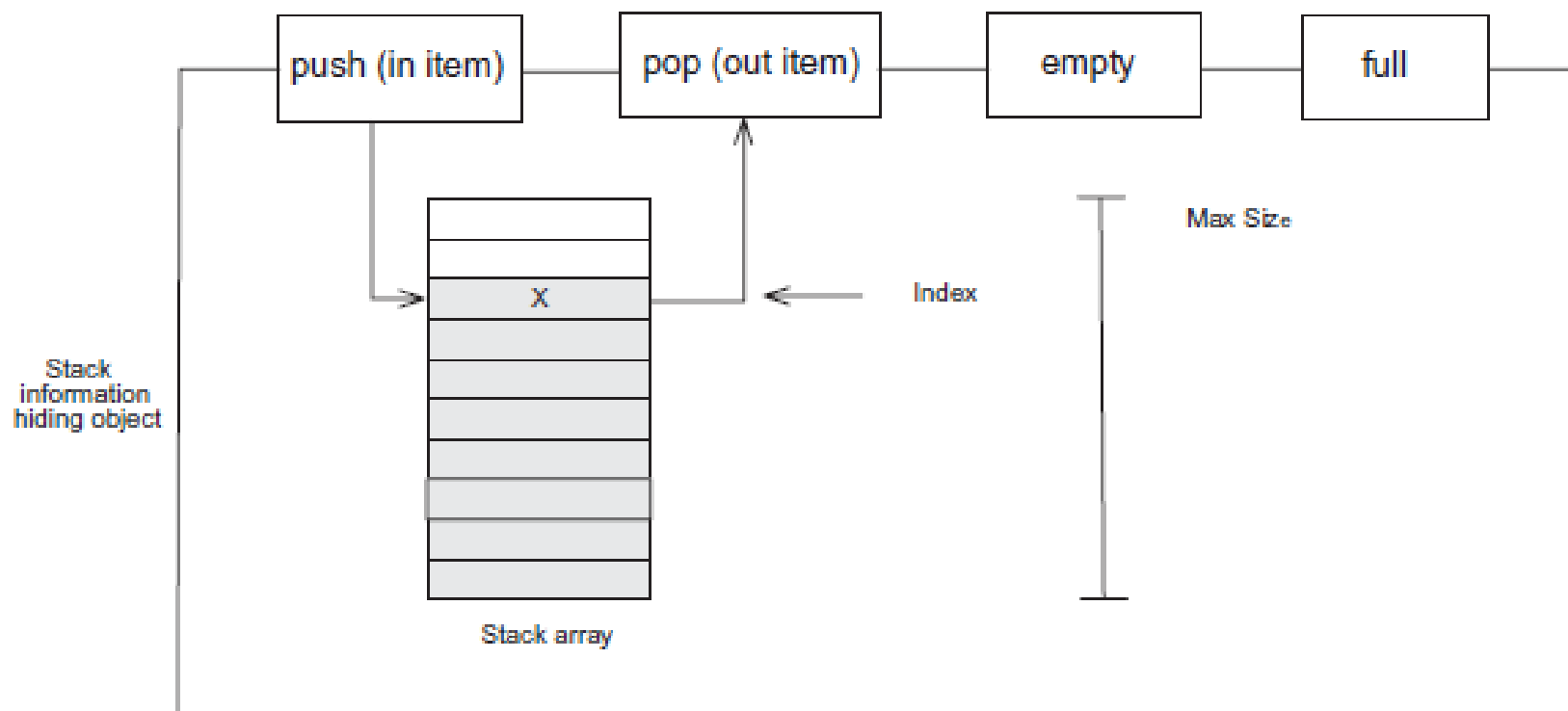
2.2 信息隐藏

- 由数组实现的非信息隐藏的栈数据结构



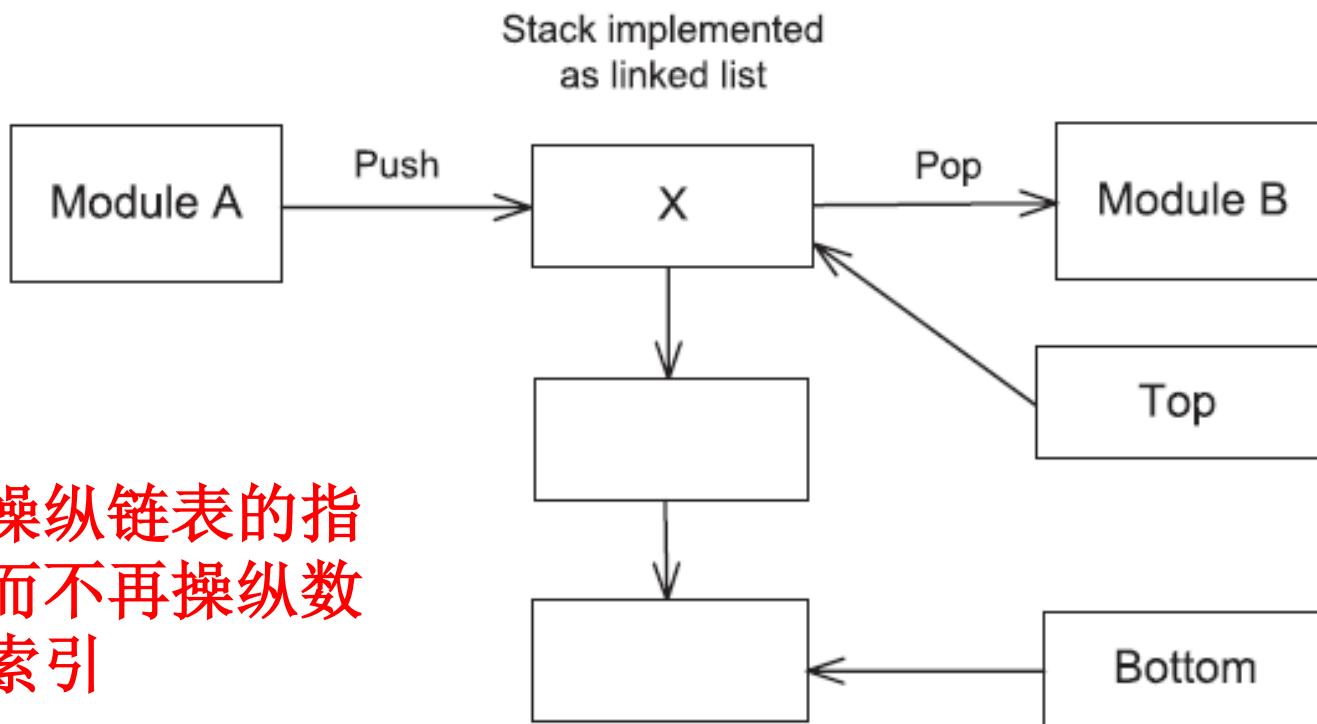
2.2 信息隐藏

- 由数组实现的栈用信息隐藏的对象来实现



2.2 信息隐藏

○ 由链表实现的非信息隐藏的栈数据结构

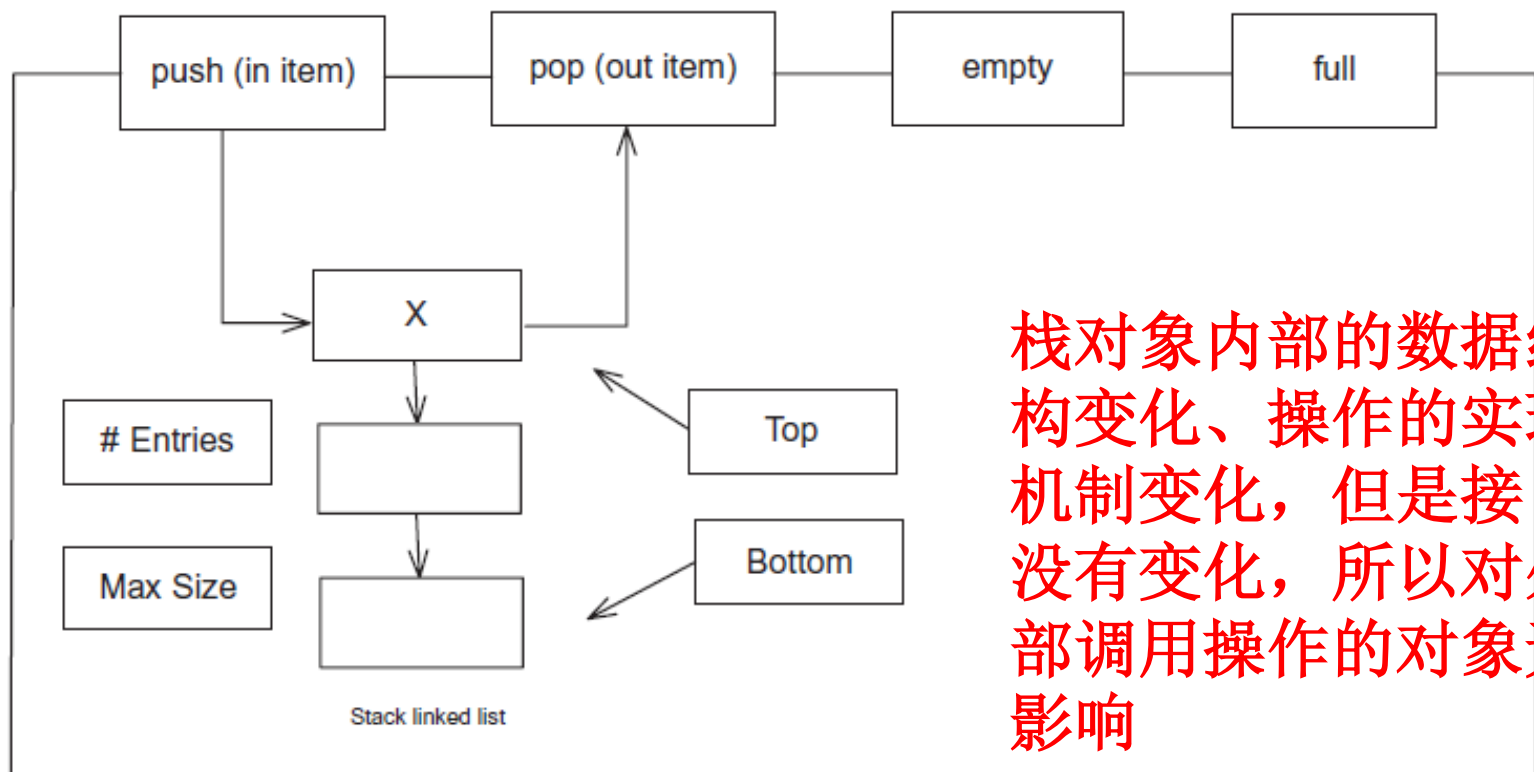


模块操纵链表的指针，而不再操纵数据的索引



2.2 信息隐藏

- 由链表实现的栈用信息隐藏的对象来实现



栈对象内部的数据结构变化、操作的实现机制变化，但是接口没有变化，所以对外部调用操作的对象无影响



2.2 信息隐藏

- 设计的栈类，作为创建栈对象的模板

Stack
push (in item) pop (out item) empty () : Boolean full () : Boolean



2.2 信息隐藏

- 如何设计信息隐藏对象

- 首先设计作为外部视图的接口 → 总体/概要设计

- 关键是给出接口中操作的规约

- 然后设计内部细节 → 详细设计

- 不要暴露出封装在对象中的所有变量

- 通过get/set操作来实现对变量的操作



2.3 继承和泛化/特化

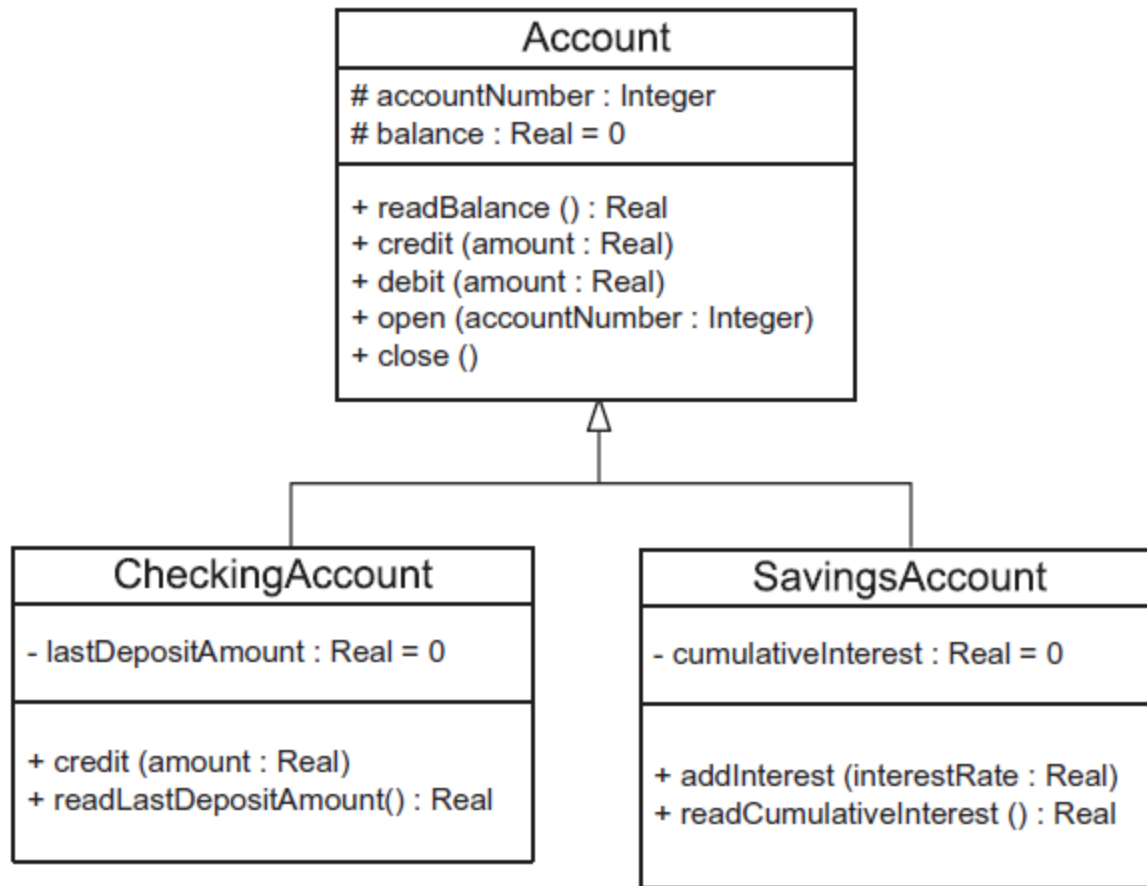
● 继承

- 一种抽象机制
- 对那些在某些方面相似但不完全相同的对象建模
 - 共同特征
 - 唯一特性
- 在不同类中分享和复用代码的机制
 - 超类/基类/父类
 - 派生类/子类
 - 将父类进行修改从而形成子类的过程称为 **特化**
 - 子类还可以被进一步特化，形成 **泛化/特化层次结构**
- 类继承实现了一种扩展机制



2.3 继承和泛化/特化

○ 例子



2.4 并发处理

● 主动对象

- 又称为：并发对象/并发过程/并发任务/线程
- 拥有自己的控制线程并能独立于其他对象而执行
- 每个主动对象处理一个顺序执行的线程
- 主动对象中不允许并发操作，整个系统的并发是通过多个主动对象的并行执行来实现
- 主动对象可以异步执行，通常独立但也可相互通信同步



2.4 并发处理

- 被动对象

- 没有控制线程
- 被主动对象调用其操作
- 可以调用其它被动对象的操作
- 其操作一旦被主动对象调用，就会在主动对象所控制的线程中执行



2.4 并发处理

- 主动对象间的协作

- 并发系统中，主动对象往往需要相互协作来完成系统功能
- 需考虑如下问题：
 - 互斥问题
 - 同步问题
 - 生产者/消费者问题



2.4 并发处理

● 同步问题

- 两个任务需在没有进行数据通信的情况下同步它们的操作
- 解决方案：事件同步（事件发送可以异步执行）
 - **Step1:** 源任务执行 **信号** 操作（发事件（可异步））
 - **Step2:** 目标任务执行 **等待** 操作
 - 目标任务线程可能被挂起



2.4 并发处理

- 同步问题示例
- 问题描述:



机器人系统：系统中每个机器人控制模块被设计为一个并发的对象并控制一个移动的机器手。

1. 执行拾取和放置的机器手(P&P)将物件抓到工作场地
2. 钻孔机器手(D)在物件上钻4个孔
3. 执行拾取和放置的机器手将物件移走

分析:

1. 存在一个碰撞区域，其中P&P可能和D发生碰撞
2. P&P必须在D开始钻孔前放好物件
3. D必须在P&P移走物件前完成钻孔



西北工业大学

NORTHWESTERN POLYTECHNICAL UNIVERSITY

2008-2024 by Li Weigang. All rights reserved.

2.4 并发处理

○ 主动对象的同步

P&P

D



西北工业大学

NORTHWESTERN POLYTECHNICAL UNIVERSITY

2008-2024 by Li Weigang. All rights reserved.

2.4 并发处理

● 生产者/消费者问题

- 生产者主动对象生产信息
- 消费者主动对象消费信息
- 由于每个主动对象拥有它自己的控制线程，独立执行，所以主动对象间希望交换数据时，它们的相关操作必须同步
 - If消费者准备接收数据，**but**生产者还未生产出数据，**then**消费者必须等待生产者
 - If消费者准备接收数据之前，生产者已经生产出数据，**then**生产者挂起等待数据被接收**or**生产者缓存数据继续生产



2.4 并发处理

- 解决方案：消息通信

- 数据传输

生产者  消费者

- 同步机制

1. 消费者等待消息

2. **if** 异步通信

then 生产者发送消息后继续执行

else if 同步通信

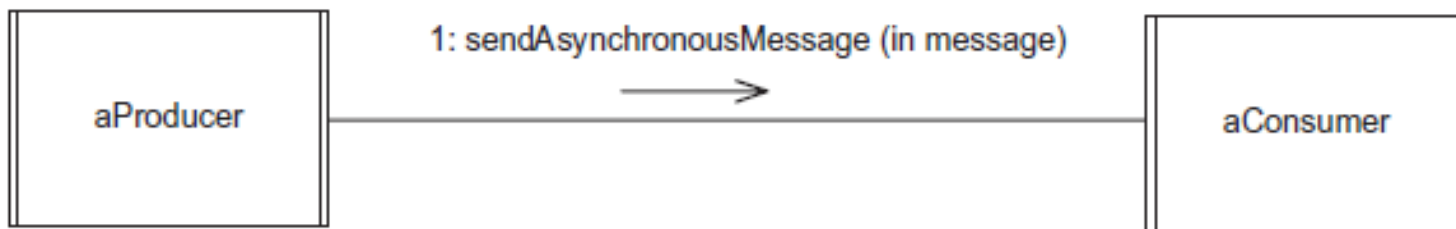
then 生产者等待回复



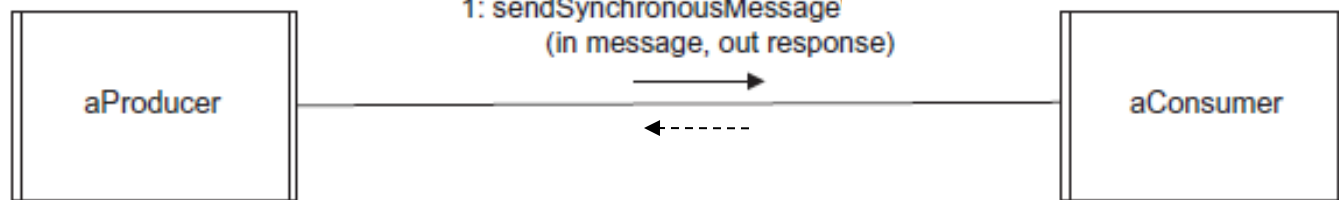
2.4 并发处理

○ 异步消息通信

- 生产者发送消息后不需要等待回复
- 消费者接收消息
- 生产者和消费者执行步调不一致
- 在生产者和消费者之间建立一个先入先出（**FIFO**）的消息队列



并发处理



○ 同步消息通信

● 生产者向消费者发送消息

1. 生产者等待回复（带回复的同步消息通信） **or**
2. 生产者等待消费者接收消息（不带回复.....）

● 消息到达时

1. 消费者接收消息，释放生产者（不带回复.....）
- or**
2. 消费者接收消息并处理，生成一个回复并发回（带回复.....）

● 然后，生产者和消费者均继续执行

● 如果没有消息，消费者挂起



2.5 设计模式

- 设计模式（**Design pattern**）是一套被反复使用、多数人知晓的、经过分类编目的设计经验的总结
- **A software design pattern is “a repeatable solution for a commonly-occurring software problem.”**
- 设计模式已被其他开发者使用、测试和改进，你不需要“重复发明轮子”



2.5 设计模式

- 设计模式涉及多个类及类对象之间的交互，所以设计模式提供了一种比类的粒度更大的复用方式

微体系结构/微架构



西北工业大学

NORTHWESTERN POLYTECHNICAL UNIVERSITY

2008-2024 by Li Weigang. All rights reserved.

2.6 软件架构和构件

- 软件架构的核心是定义构件及构件间的连接关系
- 定义构件需要描述其接口
 - 构件接口既要描述它所 **提供的操作**，又要描述它所 **需要的操作**
 - 对象接口一般仅需描述对象所 **提供的操作**
- 定义将构件结合起来的 **连接器**
 - 连接器封装了两个或多个构件之间的互连协议
 - 同步、异步

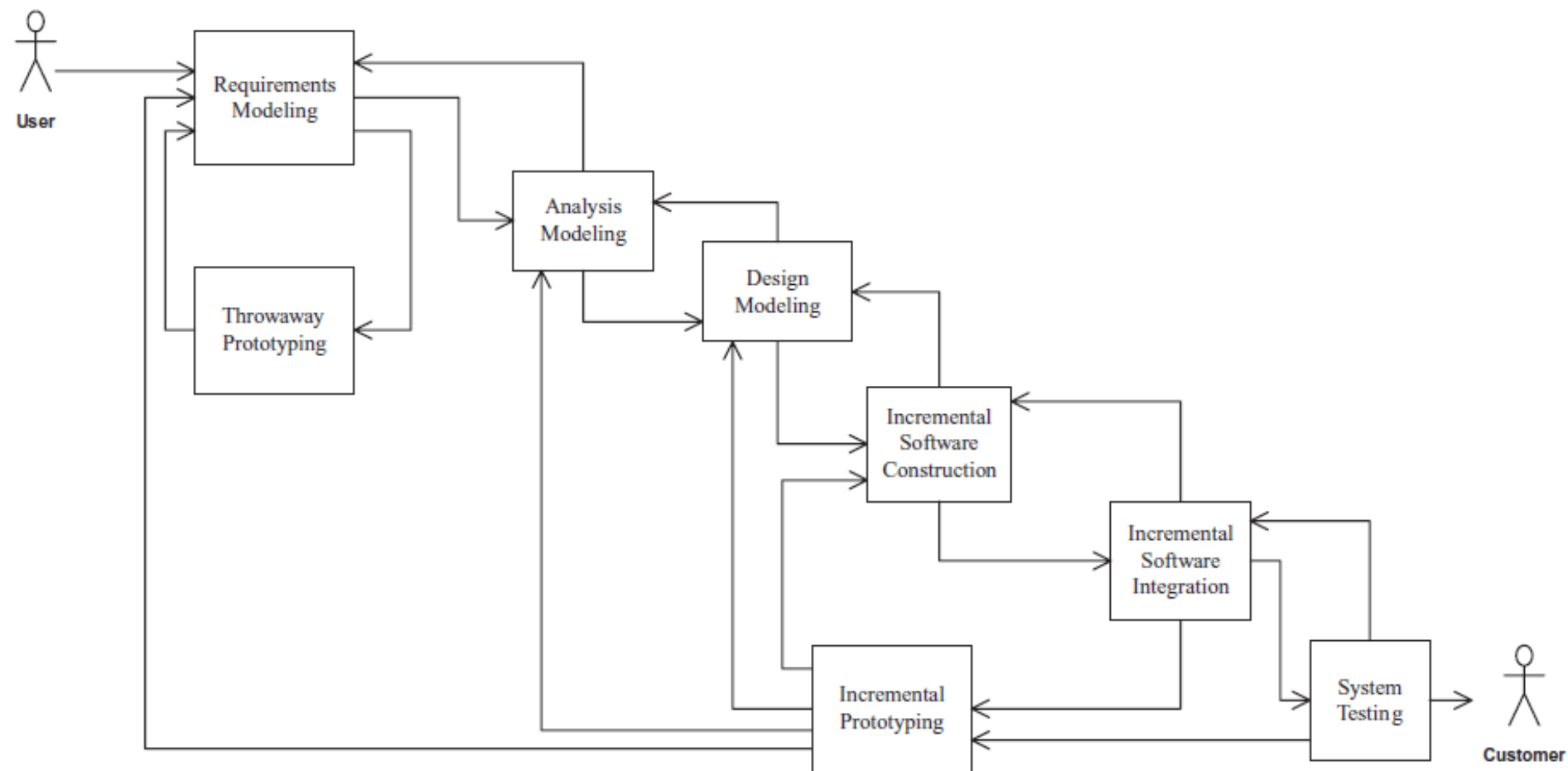


2.7 软件质量属性

- 质量属性（**QoS**）体现了软件的质量需求（非功能）
- 在设计软件架构时，考虑**QoS**非常重要
 - 性能
 - 安全性
 - 可用性
 - 可伸缩性
 - 可复用性
 - 可维护性
 - 可修改性
 - 可测试性
 - 可追踪性



2.8 基于用例的软件生存周期



2.9 需求、分析和设计建模活动

- 需求建模

- 用例建模

- 系统的功能需求采用用例和参与者来描述
 - 用例定义了一个或多个参与者与系统之间的交互序列
 - 用例描述是一个行为视图；用例间的关系给出了一个结构视图

- 陈述非功能性需求



2.9 需求、分析和设计建模活动

● 分析建模

- 主要考虑对问题域的理解

- 静态建模

- 重点是在问题域中对现实世界的类进行 **信息建模**
- 提供信息的结构视图
- 此处的类是由它们的属性及其和其他类之间的关系定义的
- 操作不是在这里定义，而是在设计模型中定义
- 对信息密集的系统，静态建模格外重要



2.9 需求、分析和设计建模活动

○对象的组织

- 确定参与每个用例的对象
- 哪些是实体对象
- 哪些是边界对象
- 哪些是控制对象
- 哪些是应用逻辑对象

○动态交互建模

- 实现用例来显示参与每个用例的对象之间的交互
- 开发通信图或序列图来显示对象如何相互通信来执行用例

○动态状态机建模

- 对状态相关的对象进行建模——状态图



2.9 需求、分析和设计建模活动

● 设计建模

- 主要考虑问题的解决方案
- 将分析模型映射为并发设计模型
 - 集成对象通信模型
 - 子系统结构和接口决策——总体架构
 - 架构模式和设计模式的决策
 - 类接口的决策，设计每个类的操作和每个操作的参数
 - 应用分布决策，分布式构件、构件通信接口
 - 对于对象特性的决策：主动/被动？
 - 消息特性的决策：同步/异步？





西北工业大学

NORTHWESTERN POLYTECHNICAL UNIVERSITY

THE END